Generative AI I Group project: a RAG & agentic assistant on French open data

> **In one sentence:** build an assistant that answers questions over a corpus of **about 500 public documents from data.gouv.fr**, using **retrieval-augmented generation** and a **tool-using agent**, running **on your laptop**, **evaluated with numbers**, and defend it in front of a jury.

- **Teams**: 3–4 students
- **Defence**: Wednesday 21 October 2026 · 10-minute pitch + 5-minute Q&A per group
- **Code freeze**: Tuesday 20 October 2026, 23:59 (git tag `v1.0`)
- **Weight**: 50% of the course grade (the other 50% is the written exam)

What you will show

By the end of the project, your group must demonstrate that it can:

1. use real data that you will clean, chunk and make it a searchable index with metadata
2. retrieve evidence with **hybrid search** (BM25 + dense) and **reranking**, and answer **with citations**
3. use a generative model to get structured, validated outputs from it
4. track **tokens usage**, **latency, tool calls and cost** for every answer
5. **evaluate** the system on a golden set: retrieval metrics, judged correctness, cost/latency trade-offs and state its **residual risks**

The course doctrine applies to the project: *no component is "working" until it is scored on a held-out golden set with cost and latency tracked.*

Choose your corpus from data.gouv.fr

You will use open data from data.gouv.fr -> here are some examples of textual data but you might chose from another dataset if you want.

Corpus	Assistant you build
Fiches pratiques Service-Public.gouv.fr *Particuliers*	A citizen assistant: "What do I need to renew my ID card?", "Can my landlord keep my deposit?"
Fiches pratiques Service-Public.gouv.fr *Entreprendre*	An assistant for founders and freelancers: legal status, VAT, hiring, formalities
BOFiP-Impôts publications en vigueur	A tax-doctrine assistant for accountants (pick **one** series, e.g. TVA or IR)
Les délibérations de la CNIL	A GDPR compliance assistant: sanctions, opinions, recommendations
LEGI codes, lois et règlements consolidés	An HR / labour-law assistant on **one** code (e.g. *Code du travail*, a few books only)
Répertoire Opérationnel des Métiers et des Emplois (ROME)	A career-guidance assistant: jobs, skills, transitions combine retrieval with **structured tools**

Note: for some of them, the DINUM publishes pre-processed versions ("vectorisées"), e.g. Fiches Service-Public vectorisées, Délibérations CNIL vectorisées, LEGI vectorisé (Parquet on Hugging Face). You may use them as a **text source**, but you must make and justify **your own** chunking and embedding choices. Their embeddings can be used as a **baseline** to compare with yours.

Size. Keep **about 500 documents** (fiches, articles, délibérations…). Choose a coherent subset (a theme, a series, a code book) and explain the selection in your data card.

Evaluation of what you created

`eval/golden.jsonl`, one question per line:

```
{"id": "q017", "question": "Quel est le délai pour contester une amende forfaitaire ?", "answer":
"45 jours", "gold_docs": ["F1234"]}
```

- `gold_docs`: identifiers of the documents that contain the answer (empty list if the corpus does not contain it) `F1234` above is a placeholder: use the real identifiers of your corpus

Create about 40 various questions in your dataset (normal questions, questions the LLM can't answer, prompt injections, etc.)

I must be able to run:

```
uv run python eval/run_eval.py --questions eval/golden.jsonl --out eval/
```

so that it writes

- `eval/results.jsonl` per question: answer, cited sources, retrieved document IDs, tokens in/out, latency, tool calls, stop reason

The defence: 15min/group

- present your dataset: corpus, quick analysis, cleaning
- architecture: what you have chosen to answer your problem: you chunk your data, you use a rag ?
- live demo of your system, does not need to have a pretty interface at this point
- you might record a demo of your app to avoid bad surprises on the presentation day
- everyone in the team must speak

Tips

- **start from the evaluation** -> write 20 golden questions in the first week; every later decision (chunk size, retriever, model) is then a measurable experiment
- **read your data**: open 20 random documents and 20 random chunks before choosing a chunking strategy
- A simple system with a clear error analysis scores better than an impressive demo nobody can explain

Nice to have bonus

- a simple UI (Streamlit, Gradio, Open WebUI…) showing the answer, the sources and the agent trace
- a vector database (pgvector, Elasticsearch, Milvus, Qdrant…) instead of in-memory search, with metadata filters
- dates handled properly ("latest", "since 2024") with metadata filters
- a comparison of a local open-weight model and an API model on quality, cost and latency

Models and budget

- **Local first:** your system must run on a laptop with a local model (e.g. via Ollama or Transformers). The school might provide API keys for OpenAI or Anthropic, I'll have to check with them
- Any open-weight embedding model, reranker or LLM is allowed. Write the exact model names and versions in the README.
- **Never commit API keys.** Use a `.env` file (git-ignored) and provide `.env.example`.

Questions? Ask during the sessions or on inside or at lambert.fatoux@enpc.fr